LINKED LIST MANAGEMENT SYSTEM

DATA STRUCTURES AND ALGORITHMS - MINI PROJECT

Language: Java

Submitted by: Mehak

System ID: 2025497969

Course: B.Tech CSE

Batch: 2

Section: E

1. INTRODUCTION

A linked list is a linear data structure in which elements, called nodes, are connected using references. Unlike an array, a linked list does not need continuous memory and can grow or shrink while the program is running. This project implements a menu-driven console application in Java that manages a Singly Linked List and a Doubly Linked List.

2. OBJECTIVE

• Implement a Singly Linked List and a Doubly Linked List manually using node objects.

• Implement all required operations as separate methods.

• Avoid arrays and built-in list collections for the list logic.

• Provide a menu-driven interface for both lists and a comparison screen.

• Test all important edge cases.

3. LANGUAGE CHOICE

Java was selected. Object references replace pointers, memory is managed by the garbage collector (no explicit delete), and input and output are handled using Scanner and System.out.

4. DESIGN EXPLANATION

Singly Linked List. Each node holds data and a next reference. The list stores only head.

```
head -> [10 | next] -> [20 | next] -> [30 | NULL]
```

Doubly Linked List. Each node holds prev, data and next. The list stores head and tail, which makes insertion and deletion at the end O(1) and allows backward traversal.

```
NULL <- [10] <-> [20] <-> [30] -> NULL
        ^head               ^tail
```

Classes.

CLASS	RESPONSIBILITY
SinglyLinkedList	Private Node class, all singly list operations and the Singly menu
DoublyLinkedList	Private Node class (prev, data, next), head and tail, all doubly list operations and the Doubly menu
Main	Main menu, comparison screen, safe number input (readInt)
EdgeCaseTests	Runs 92 automatic checks covering every required edge case

Rules used. Positions are 1-based. Insert at Position accepts 1 to count+1; Delete from Position accepts 1 to count. Operations by value act on the first match. For even-length lists, Find Middle returns the second middle node. Remove Duplicates keeps the first occurrence.

5. OPERATION LOGIC (SHORT EXPLANATION)

OPERATION	IDEA
Insert at Beginning	New node's next = head; head = new node.
Insert at End	Singly: walk to the last node and attach. Doubly: attach after tail and update tail.
Insert at Position	Validate 1..count+1; stop at the node before the position; relink.
Insert After / Before	Find first node with the target value; relink neighbouring nodes.
Delete (begin / end / position / value)	Check empty; bypass the node by connecting its neighbours; handle head and tail separately.
Search / Update	Traverse from head, compare each value, report the 1-based position.
Find Max / Min	Traverse once, keep the best value seen so far.
Find Middle	slow moves 1 step, fast moves 2 steps; when fast ends, slow is at the middle.
Reverse (singly)	Use previous, current, next to flip every next link; head = previous.
Reverse (doubly)	Swap prev and next in every node; swap head and tail.
Sort	Bubble Sort: swap data of neighbouring nodes; the largest value reaches its place each pass; stops early when no swap happens.
Remove Duplicates	Two loops, no extra collection: for every node, unlink later nodes with the same value.

6. TIME COMPLEXITY

OPERATION	SINGLY	DOUBLY
Insert at Beginning	O(1)	O(1)
Insert at End	O(n)	O(1)
Insert at Position	O(n)	O(n)
Insert After / Before a Node	O(n)	O(n)
Delete from Beginning	O(1)	O(1)
Delete from End	O(n)	O(1)
Delete from Position / by Value	O(n)	O(n)
Search / Display / Count	O(n)	O(n)
Find Max / Min / Middle	O(n)	O(n)
Reverse	O(n), space O(1)	O(n), space O(1)
Bubble Sort	O(n^2)	O(n^2)
Remove Duplicates	O(n^2)	O(n^2)

7. SINGLY VS DOUBLY LINKED LIST

FEATURE	SINGLY	DOUBLY
Links per node	next	prev and next
Memory per node	Less	More
Traversal	Forward only	Forward and backward
Insert / delete at end	O(n)	O(1) with tail
Code complexity	Simpler	More links to maintain

8. TESTING AND EDGE CASES

The class EdgeCaseTests checks both lists automatically. Doubly list tests verify both forward and backward output so that broken prev links are detected. Result of the final run: 92 tests, 92 passed, 0 failed.

EDGE CASE	EXPECTED BEHAVIOUR
Empty list	Displays NULL; delete, max, min, middle report empty; no crash
One-node list	Works correctly; deleting leaves an empty list
Insert / delete at position 1	Head is updated correctly
Insert at last valid position	Position count+1 appends the node
Invalid / out-of-range position	Operation refused; list unchanged
Delete a value that does not exist	Reports not found; list unchanged
Duplicate values	First match used; Remove Duplicates keeps the first occurrence
Zero and negative values	Max, min and sort give correct results
Sort empty / single-node list	No change, no crash
Reverse empty / single-node list	No change, no crash

9. HOW TO RUN

```
javac -d out src/*.java
java -cp out Main
java -cp out EdgeCaseTests
```

10. CONCLUSION

The project successfully implements both a Singly and a Doubly Linked List with all required operations, input validation and complete edge-case testing. It demonstrates how references connect nodes, how the choice of extra references (tail, prev) changes time complexity, and why linked lists are suitable for frequent insertion and deletion.